

データ分析ゼミ：環境構築

はじめての開発環境と Markdown

授業用教材・試作版

目次

この教材について	5
到達目標	5
授業での使い方	5
1 環境構築	7
1.1 作業前の準備	7
1.2 macOS: Homebrew · VS Code · Git	7
1.3 Windows: winget · VS Code · Git	8
1.4 演習: Markdown を書いてプレビューする	9
1.5 GitHub アカウントを準備する	10
1.6 Git の初期設定	11
1.7 GitHub Education を申請する	11
1.8 完了チェックと提出	12
1.9 困ったとき	12
2 Python の基礎	15
3 Python の基礎	17
3.1 この章の目標と進め方	17
3.2 準備: ノートブックを開く	17
3.3 計算と変数	19
3.4 数値 · 文字列 · 真偽値	20
3.5 リスト: 複数の値をまとめる	21
3.6 条件分岐: 条件で処理を変える	21
3.7 繰り返し: 同じ処理を各データに行う	22
3.8 辞書: 項目名と値を対応させる	22
3.9 関数: 処理に名前を付ける	23
3.10 エラーを読んで直す	23
3.11 総合課題: 学習時間の集計メモ	24
3.12 解答例と振り返り	24
3.13 公式資料	26

この教材について

プログラミングを始めるための道具をそろえ、VS Code で最初の学習メモを作成します。対象は、データ分析をこれから学ぶ大学生です。

到達目標

- 自分の OS に合った方法で VS Code と Git を導入できる。
- Markdown ファイルを編集し、整形されたプレビューを表示できる。
- GitHub アカウントを準備し、GitHub Education を申請できる。

授業での使い方

所要時間は 75 ～ 105 分 が目安です。ダウンロード時間と Education の審査期間により前後します。授業全 15 回・サブゼミ全 10 回のうち、初回の環境構築に使い、申請のフォローをサブゼミで行う想定です。

作業	目安
導入・OS 確認	5 分
VS Code・Git の導入	25 ～ 40 分
Markdown 演習	15 分
GitHub・Git 設定	15 ～ 20 分
Education 申請・完了確認	15 ～ 25 分

自分の OS の節だけを実施し、その後は共通の演習へ進みます。既に導入済みのソフトは、バージョンを確認できれば先へ進めます。

この章では Python・Jupyter・Quarto のインストールは扱いません。Markdown 演習は VS Code の標準機能だけで行えます。

試作版：2026 年 9 月 15 日確認。画面や申請条件が変わった場合は、各節の公式資料を参照してください。Web 版と PDF 版は同じ Quarto 原稿から生成しています。

1. 環境構築

1.1 作業前の準備

自分が管理できる PC、安定したインターネット接続、受信できる個人メールと大学メール、現在の在学状況を確認できる学生証などを用意します。大学管理の PC では、ソフトの導入可否を担当者に確認してください。

1.1.1 道具の役割

道具	何に使うか
Homebrew / winget	アプリをコマンドで導入・更新するパッケージマネージャ
VS Code	文章やプログラムを書くエディタ
Git	ファイルの変更履歴を記録するソフト
GitHub	Git で管理した資料を共有する Web サービス
Markdown	見出しや箇条書きを簡単な記号で書く形式

「コマンド」は PC への文字による指示です。Mac ではターミナル、Windows では PowerShell に入力して Enter を押します。コード枠の中だけをコピーし、複数行は上から 1 行ずつ実行します。行頭に表示されるプロンプト（\$ や PS C:\...>）は入力しません。

パスワード・認証コード・回復コードは、教材のメモや提出物に書かないでください。

1.2 macOS: Homebrew・VS Code・Git

Windows の人は [Windows の手順](#)へ進みます。

1.2.1 OS とターミナルを確認する

1. Apple メニューの「この Mac について」を開き、macOS のバージョンとチップを確認します。「Apple M...」なら Apple Silicon、「Intel」なら Intel Mac です。
2. [Homebrew の動作要件](#)を確認します。対応外の OS の場合は担当者へ相談します。
3. Spotlight（Command+Space）で「ターミナル」を検索し、起動します。

1.2.2 Homebrew を導入する

まず次を実行します。バージョンが表示されれば、インストール作業は省略できます。

```
brew --version
```

未導入なら [Homebrew 公式サイト](#)のインストールコマンドを確認して実行します。以下は確認時点のコマンドです。折り返されていても 1 行です。

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

これは公式のインストール用スクリプトを取得し、実行する指示です。画面に出る変更内容を読んで進めます。パスワードを求められたら Mac へのログインパスワードを入力します。入力中に

文字や丸印が表示されなくても正常です。Command Line Tools の導入が始まった場合は完了を待ちます。

完了画面の **Next steps** にコマンドが表示されたら、その指示を実行します。これは Homebrew を探す場所 (PATH) をターミナルに知らせる設定です。通常の導入先は Apple Silicon が `/opt/homebrew`、Intel が `/usr/local` ですが、自分の画面に表示された指示を使ってください。

ターミナルを開き直し、確認します。

```
brew --version
brew doctor
```

前者でバージョン、後者で診断結果が表示されれば実行できています。警告がある場合は内容を確認します。警告すべてが授業を止める問題とは限りません。判断できない場合はエラー文を担当者に見せます。

1.2.3 VS Code と Git を導入する

```
brew install --cask visual-studio-code
brew install git
```

完了後、アプリケーションから Visual Studio Code を起動します。ターミナルを開き直して確認します。

```
code --version
git --version
```

両方でバージョンが出れば完了です。code が見つからない場合は、VS Code で Command+Shift+P を押し、Shell Command: Install 'code' command in PATH を検索して実行し、ターミナルを開き直します。

公式資料: [VS Code の Homebrew パッケージ](#)、[VS Code の macOS 設定](#)。

1.3 Windows: winget · VS Code · Git

Mac の人は [Markdown 演習](#) へ進みます。

1.3.1 PowerShell と winget を確認する

1. 設定の「システム」→「バージョン情報」で Windows のバージョンを確認します。大学の方針に従い、サポートされている OS を使います。
2. スタートメニューで「PowerShell」を検索して、通常の権限で起動します。Windows Terminal を使う場合は PowerShell のタブを選びます。
3. 次のコマンドを実行します。

```
winget --version
```

v で始まるバージョンが出れば利用できます。見つからなければ Microsoft Store で Microsoft 提供の「アプリ インストーラー (App Installer)」をインストールまたは更新し、PowerShell を開き直します。利用できない場合は [winget 公式資料](#) を確認します。

1.3.2 VS Code と Git を導入する

次のコマンドを 1 行ずつ実行します。

```
winget install --id Microsoft.VisualStudioCode -e --source winget
winget install --id Git.Git -e --source winget
```

--id はアプリの識別名、-e は完全一致、--source winget は検索する配布元の指定です。利用条件やインストール確認が出たら内容を読んで進みます。管理者の許可を求められた場合は、自分が開始したインストールか確認してください。管理された PC では担当者に依頼します。

インストール終了後、PowerShell を閉じて開き直します。

```
code --version
git --version
```

両方でバージョンが出れば完了です。スタートメニューから Visual Studio Code を起動できることも確認します。

公式資料: [winget install](#)、[VS Code の Windows 設定](#)。

1.4 演習: Markdown を書いてプレビューする

目安は 15 分です。完成時には、左に Markdown の原稿、右に整形された表示が並びます。

1.4.1 作業フォルダとファイルを作る

Finder またはエクスプローラーで自分のホームフォルダ（ユーザーフォルダ）を開き、seminar-work というフォルダを作成します。同名のフォルダがあれば、その中を確認して使います。

VS Code の「ファイル」→「フォルダーを開く」から、そのフォルダを選びます。自分で作ったフォルダに対する信頼確認が出たら、内容を確認して信頼します。

コマンドでも開けます。フォルダ作成後、Mac のターミナルでは次を実行します。

```
cd ~/seminar-work
code .
```

Windows の PowerShell では次を実行します。

```
cd "$env:USERPROFILE\seminar-work"
code .
```

. は「今いるフォルダ」です。VS Code の左側のエクスプローラーから「新しいファイル」を選び、setup-check.md と名付けます。

1.4.2 Markdown を入力する

次の内容を入力またはコピーします。コード枠の外側のバッククォートは入力しません。

```
# 環境構築チェック

## 使用環境

- OS : macOS または Windows
- エディタ : VS Code
- バージョン確認 : `git --version`

## 完了した作業

- [ ] VS Code を起動した
- [ ] Git のバージョンを確認した
- [ ] GitHub アカウントを作成した

## 参考資料

- [GitHub Docs](https://docs.github.com/)
```

Mac は Command+S、Windows は Ctrl+S で保存します。ファイル名が setup-check.md.txt になっていないことを確認してください。

記号	表示の意味
#、##	見出し、ひとつ下の階層の見出し
-	箇条書き
- [], - [x]	未完了、完了のチェック項目
バッククォートで囲む	コマンドなどを本文と区別して表示
[表示名](URL)	リンク

や - の後には半角スペースを入れます。段落やリストの前後には空行を入れます。

1.4.3 通常プレビューを開く

原稿のタブを選び、次のショートカットを押します。

操作	macOS	Windows
プレビュー	Command+Shift+V	Ctrl+Shift+V
横にプレビュー	Command+K → V	Ctrl+K → V

横に表示するときは、原稿へ戻ってから操作します。矢印は「最初のキーの組み合わせを押して離し、続けて V を押す」という意味です。

メニューから開く場合は、コマンドパレット (Mac: Command+Shift+P、Windows: Ctrl+Shift+P) で `Markdown: Open Preview to the Side` を検索します。

1.4.4 編集が反映されるか確認する

1. 左の原稿で OS 名を自分の環境に合わせて書き換えます。
2. VS Code 起動の項目を - [x] `VS Code` を起動した に変更します。
3. ## 今日わかったこと を追加し、その下に学んだことを 1 行書きます。
4. 右のプレビューで見出し・チェック状態・文章が更新されたか確認します。
5. 保存して、ファイルを開き直しても変更が残ることを確認します。

演習の成功条件: 原稿とプレビューを左右に表示でき、編集結果が反映され、保存したファイルが残っていることです。チェック項目は原稿側で書き換えます。GitHub の項目は後の作業が終わってから更新します。

この演習では VS Code 標準の Markdown プレビューを使います。追加の拡張機能は不要です。プレビューのセキュリティ設定は既定の Strict のままで表示できます。

公式資料: [Markdown 編集とプレビュー](#)。

1.5 GitHub アカウントを準備する

1.5.1 アカウントとメール認証

既に個人アカウントがある人は、それを利用します。

1. [GitHub](#) を開き、Sign up から個人アカウントを作成します。
2. 継続して使えるユーザー名を選びます。ユーザー名は公開プロフィールや URL に使われます。
3. 画面の案内に従い、登録メールに届く確認を完了します。
4. サインイン後、Settings → Emails で大学メールを追加します。
5. 大学メール宛での確認を完了し、認証済みになっていることを確認します。

卒業後も使う個人メールを登録しておく、大学メールが失効した後も連絡先を維持できます。公開プロフィールにメールを表示するかどうかは別に設定できます。

公式資料: [アカウントの作成](#)。

1.5.2 二要素認証を設定する

二要素認証（2FA）は、パスワードに加えて認証アプリなどで本人確認を行う仕組みです。

1. Settings → Password and authentication を開きます。
2. Two-factor authentication の設定を開始し、認証アプリなどの方法を画面に従って登録します。
3. 回復コードを保存します。スマートフォンを紛失しても使えるよう、パスワード管理アプリなど安全な場所に保管します。
4. 設定画面で二要素認証が有効であることを確認します。

回復コードはアカウントへのアクセスを回復するための秘密情報です。GitHub のリポジトリ、`setup-check.md`、授業の提出物には保存しません。

公式資料: [二要素認証の設定](#)。

1.6 Git の初期設定

Git は変更履歴に作成者の名前とメールを記録します。この設定は GitHub へのログインとは別です。公開する履歴にも含まれるため、設定する値を先に決めます。

GitHub の Settings → Emails でメール非公開の設定を確認すると、GitHub 提供の `noreply` アドレスを確認できます。個人のメールを公開したくない場合は、そのアドレスをそのままコピーして使います。文字列を推測して作らないでください。

次の引用符内を自分の値に置き換えてから、ターミナルまたは PowerShell で実行します。名前は履歴に表示したい名前、メールは GitHub で認証済みのメールか GitHub 提供の非公開用メールにします。

```
git config --global user.name "自分の表示名"
git config --global user.email "自分のメールアドレス"
git config --global init.defaultBranch main
```

`--global` はこの PC の自分のユーザー全体に適用する指定です。既に設定済みなら、変更前に下の確認コマンドで現在の値を見てください。main は新しく履歴管理を始めるときの既定ブランチ名です。

```
git config --global --get user.name
git config --global --get user.email
git config --global --get init.defaultBranch
```

指定した名前・メール・main が順に表示されれば完了です。全設定を自分で確認するには `git config --global --list` を使えますが、出力全体を提出する必要はありません。接続設定、コミット、push は後の授業で扱います。

公式資料: [Git の初期設定](#)、[コミットのメール設定](#)。

1.7 GitHub Education を申請する

1.7.1 条件と書類を確認する

申請には個人の GitHub アカウントが必要です。学生向けの主な条件は 13 歳以上で、学位・卒業資格につながる教育課程に現在在籍していることです。学校発行の認証可能なメール、または現在の在籍を証明する資料を用意します。学校によって大学メールの認証が必要です。

在学資料には、氏名・学校名・現在の在籍期間が確認できる学生証、履修時間割、成績証明書、在学証明書などを使います。日付のない学生証しかない場合は、現在の在籍が分かる追加資料を準備します。

1.7.2 申請を送信する

1. GitHub へサインインします。
2. [Education benefits](#) を開き、申請開始（Start an application）を選びます。

3. 学生として申請し、学校名・大学メールなど、フォームが求める情報を入力します。
4. 在学証明を求められたら、文字と日付が読める資料をフォームの案内に沿って提出します。
5. 内容を確認して送信し、申請状況を確認します。

ここでの完了は申請送信までです。審査待ちは環境構築の失敗ではありません。`setup-check.md`に「申請済み・審査待ち」などの状態だけ記録します。証明書の画像は学習メモや公開リポジトリに入れません。

大学メールが発行されない学校では、申請画面の案内に沿って在学資料や学校のメール発行方針を示します。ドメインが認識されない場合は公式ヘルプの案内からサポートへ連絡します。

条件・申請方法の一次情報: [GitHub Education への学生申請](#)。

1.7.3 承認後の確認

承認されたら Education benefits で利用できる特典を確認します。GitHub Copilot などの特典は別途有効化が必要な場合があり、反映に数日かかることもあります。有料プランしか表示されない場合は、そのまま購入せず、承認状態と公式案内を確認します。

公式資料: [学生向け Copilot の設定](#)。

1.8 完了チェックと提出

次の項目を自分で確認し、`setup-check.md`を更新します。

- ☐ `brew --version` または `winget --version` が表示される。
- ☐ VS Code を起動でき、`code --version` が表示される。
- ☐ `git --version` が表示される。
- ☐ `setup-check.md` を作成・保存できる。
- ☐ Markdown の通常プレビューと左右分割プレビューを開ける。
- ☐ 原稿の編集がプレビューに反映される。
- ☐ Git の名前・メール・既定ブランチを確認した。
- ☐ GitHub にサインインでき、登録メールと大学メールを認証した。
- ☐ 二要素認証を有効にし、回復コードを安全に保管した。
- ☐ Education を申請した。または、未申請の理由を整理した。

授業での提出例は `setup-check.md` です。OS、確認したソフトのバージョン、完了項目、分かったこと、困っていることを記録します。アカウント名・メール・認証情報・学生証は含めません。提出先は授業担当者が指定します。

確認問題：Git と GitHub はそれぞれ何をする道具ですか。Markdown の原稿とプレビューはどのような関係ですか。自分の言葉で説明してみましょう。

1.9 困ったとき

1.9.1 Mac で brew が見つからない

導入後の PATH 設定が未反映の可能性があります。インストール最後の Next steps を確認し、指示を実行してターミナルを開き直します。[Homebrew 導入資料](#)を参照してください。

1.9.2 code または git が見つからない

インストール後も古いターミナルを使っている可能性があります。VS Code 内のターミナルも含めて開き直します。Mac の `code` は [macOS の節](#)の PATH 設定を確認します。Windows はインストール完了を確認し、[公式設定手順](#)を参照します。

1.9.3 Windows で winget が見つからない

App Installer が未導入・未更新の可能性があります。Microsoft Store で確認します。Store が利用制限されている場合は、大学の管理担当者へ相談します。[winget 公式資料](#)を参照してください。

1.9.4 Markdown プレビューが開かない・更新されない

ファイルの拡張子が `.md` か確認します。原稿のタブを選び、コマンドパレットから Markdown のプレビューを開きます。別ファイルのプレビューを見ていないか確認し、保存します。横に開くキーは同時押しを 2 段階に分けます。[VS Code 公式資料](#)も確認してください。

1.9.5 GitHub の確認メールが届かない

迷惑メール、メールアドレスの入力間違い、大学メールの受信制限を確認します。Settings → Emails で再送できます。[メール認証の公式案内](#)を参照してください。

1.9.6 Education で学校が見つからない・証明が受理されない

大学メールの認証、学校名、書類の氏名・日付・可読性を確認します。却下理由が表示されたらそれを読み、必要な資料をそろえて再申請します。解決しない場合は[学生申請の公式案内](#)からサポートへ進みます。

担当者へ相談するときは、OS、実行した操作、エラー文、試した対処を伝えます。スクリーンショットに秘密情報が写っていないか確認してから共有します。

2. Python の基礎

3. Python の基礎

3.1 この章の目標と進め方

配布ファイル: [演習ノートブック](#) / [解答ノートブック](#) / [PDF 版](#)。

Python は、計算やデータ処理の手順を文章のようなコードで書く言語です。この章では、架空の 5 人の学習時間を使い、集計して説明するところまで進みます。プログラミングの経験は必要ありません。

- ・ 値に名前を付け、数値と文字列を区別する。
- ・ リストと辞書で複数のデータを扱う。
- ・ 条件分岐・繰り返し・関数を組み合わせる。
- ・ エラーを読み、上から再実行できるノートブックを作る。

授業全 15 回・サブゼミ全 10 回のうち、授業 2 回（各 90 分を**仮定**）+ サブゼミ 1 回（60 分）を使う案です。授業時間に合わせて途中の演習を宿題にできます。

回	内容	配分の目安
授業 A	実行環境・計算・変数・リスト	準備 15 分、説明と実行 45 分、演習 20 分、振り返り 10 分
授業 B	条件分岐・繰り返し・辞書・関数	復習 10 分、説明と実行 55 分、演習 20 分、振り返り 5 分
サブゼミ	エラー修正・総合課題・相互確認	修正 15 分、課題 30 分、確認 15 分

各例は「結果を予想する → 実行する → 数値を一つ変える → 違いを説明する」の順で進めてください。解答例は章末にまとめています。

3.2 準備：ノートブックを開く

ノートブックは、説明文・コード・実行結果を一緒に保存するファイルです。拡張子は `.ipynb` です。

この章では、インストール済みの `uv` と `VS Code` を使い、自分の PC で Python を実行します。`uv` は Python とパッケージを管理する道具です。初回の環境準備にはネット接続が必要です。

3.2.1 `uv` で授業用の環境を作る

Windows では、`uv` をインストールした `WSL` の Linux 環境を使います。VS Code 本体は Windows 側に置き、Microsoft の `WSL 拡張 (ms-vscode-remote.remote-wsl)` をインストールしてください。Mac の人は通常のターミナルを使い、`WSL` の操作を省略します。

Windows Terminal で Ubuntu など、`uv` を入れたディストリビューションのタブを開きます。PowerShell から入る場合は `wsl` を実行します。複数ある場合は `wsl -l -v` で名前を確認し、`wsl -d Ubuntu` のように対象を指定します（Ubuntu は自分の名前に置き換えます）。以降の `uv` コマンドは `WSL` 内のターミナルで実行します。

```
cd ~
uname -s
uv --version
```

WSL では `uname -s` が `Linux`、Mac では `Darwin` を表示します。`uv --version` でバージョンが表示されることを確認します。WSL の `~` は Linux 側のホーム（通常 `/home/ユーザー名`）です。この授業のフォルダはここに作ります。Windows の `C:` ドライブ側にある既存の `.venv` は流用しません。

続けて以下を 1 行ずつ実行します。**Python** のセルではなく、ターミナルに入力します。

```
uv init --python 3.12 python-basics
cd python-basics
uv add --dev ipykernel
uv run python --version
code .
```

新しい `python-basics` フォルダを作り、Python 3.12 を指定します。必要なら `uv` が Python を取得します。`ipykernel` はノートブックから Python を実行するためのパッケージです。最後の `.` は現在のフォルダを意味します。

WSL から `code .` を実行すると、Windows 側の VS Code が WSL に接続して開きます。初回は VS Code Server の準備を待ち、左下に **WSL: Ubuntu** など接続先の表示があることを確認します。`code` が使えない場合は、Windows 側の VS Code で `F1` を押し、**WSL: Connect to WSL using Distro** から `uv` を入れた環境を選び、そのウィンドウで Linux 側の `python-basics` フォルダを開きます。Mac の場合は通常の「フォルダーを開く」で構いません。

同名のフォルダがある場合は内容を確認してください。既に `uv` で初期化済みなら `uv init` は繰り返さず、そのフォルダへ移動して残りの手順を行います。

ファイル・フォルダ 役割

<code>.venv/</code>	授業用の Python 環境。他の作業とパッケージを分ける
<code>pyproject.toml</code>	プロジェクトと必要なパッケージの設定
<code>uv.lock</code>	使用するパッケージのバージョンの記録
<code>.python-version</code>	Python のバージョン指定

2 回目以降は同じフォルダを開きます。別の PC で設定ファイルを受け取った場合は、そのフォルダで `uv sync` を実行して環境を準備できます。`.venv` 自体をコピーする必要はありません。

3.2.2 VS Code でカーネルを選ぶ

1. WSL 接続中の VS Code で、Microsoft 提供の **Python** (`ms-python.python`) と **Jupyter** (`ms-toolsai.jupyter`) を確認します。「WSL: Ubuntu にインストール」などが表示されたら押し、**WSL 側で利用できる状態**にします。Windows 側だけへの導入では足りない場合があります。Mac では通常どおり導入します。
2. **演習ノートブック**をダウンロードします。
3. 作成した `python-basics` フォルダに保存します。Windows でダウンロードした場合は、WSL の作業フォルダで `explorer.exe` を実行し、開いたエクスプローラーに `.ipynb` をコピーします。その後、**WSL 接続中の VS Code** のファイル一覧から開きます。提出時この方法で保存済みファイルを取り出せます。
4. 右上の「カーネルの選択」から、必要なら「別のカーネルを選択」→「Python 環境」を開き、このフォルダの `.venv` を選びます。
5. **WSL** でも **Mac** でも `.venv/bin/python` を選びます。WSL では `/home/ユーザー名/python-basics/.venv/bin/python` のようなパスが目印です。Windows 側の `python.exe` ではありません。

カーネルはセルを実行するプログラムです。ターミナルで仮想環境を有効化する操作は、この手順では不要です。最初に新しいコードセルで次を実行し、パスに `python-basics` と `.venv` が含まれることを確認します。

```
import sys
print(sys.executable)
```

パスは PC ごとに異なります。次に挨拶のセルを選び、左の実行ボタン、または Shift+Enter で実行します。

「セル」はノートブックの一つの入力枠です。コードセルに Python、テキストセルに Markdown を書きます。教材 Web ページ上のコードは、その場では実行されません。

```
print("こんにちは、Python")
```

結果： こんにちは、Python と表示されます。print() は括弧内の値を表示する関数です。引用符は文字列の境界を表します。

i 実行順序が大切

上のセルで作った変数を下のセルで使います。画面の並びではなく、実際に実行した順番で状態が変わります。作業の最後はノートブック上部の「カーネルの再起動」を実行し、「すべて実行」で上から動くか確認します。再起動すると変数は消えますが、保存したコードは残ります。Mac は Command+S、Windows は Ctrl+S で保存します。

3.2.3 環境がうまく動かないとき

- **.venv** が候補に出ない: 左下に WSL 接続先が表示され、Python・Jupyter 拡張が WSL 側に入り、Linux 側の python-basics フォルダを開いているか確認します。必要ならその WSL で code . を実行し直します。Mac では開いているフォルダを確認します。
- **uv** が見つからない: PowerShell でなく、uv を導入した WSL のターミナルか確認します。別のディストリビューションに入っていないかも確認し、ターミナルを開き直します。
- **ipykernel** がない: そのフォルダのターミナルで uv add --dev ipykernel を実行し、カーネルを選び直します。
- **違う Python が動く**: セルの sys.executable と、ターミナルの uv run python -c "import sys; print(sys.executable)" のパスを比べ、同じ .venv を選択します。
- 文字として表示される: セルの種類を Markdown からコードに切り替えます。

公式資料: [VS Code の WSL 接続](#)、[uv と VS Code の連携](#)、[VS Code のノートブック操作](#)。

3.3 計算と変数

3.3.1 計算を表示する

```
print(1200 + 800)
print(1200 * 3)
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(2 ** 3)
```

結果（上から）: 2000、3600、3.5、3、1、8。

* は掛け算、/ は割り算、// は商を小さい整数側へ丸める除算、% は余り、** は累乗です。掛け算・割り算は足し算より先に計算されます。順序を変えたいときは括弧を使います。

3.3.2 値に名前を付ける

```
price = 1200
quantity = 3
total = price * quantity
print(total)
```

結果： 3600。price などを変数と呼びます。= は「右側を計算し、左側の名前に結び付ける」という代入です。数学の等式とは意味が異なります。

```
quantity = 4
print(total)
total = price * quantity
print(total)
```

結果：3600、4800。変数を変えても、過去に計算した値は自動更新されません。計算し直す必要があります。

変数名には `study_hours` のように意味の分かる英小文字とアンダースコアを使います。大文字・小文字は区別されます。`print`、`sum`、`list` のような組み込み機能の名前は変数名に使わないでください。

3.3.3 演習 1: 予算を計算する

1冊 850 円の本を 3 冊買い、送料が 200 円かかります。価格・冊数・送料をそれぞれ変数にして合計を求めてください。その後、冊数を 4 に変更して再計算します。結果が変わる理由を 1 文で説明してください。

3.4 数値・文字列・真偽値

型	例	用途
<code>int</code>	3	整数
<code>float</code>	2.5	小数を含む数値
<code>str</code>	"ゼミ"、"3"	文字列
<code>bool</code>	<code>True</code> 、 <code>False</code>	条件が成り立つか

```
hours_text = "3"
print(hours_text + "2")
hours = int(hours_text)
print(hours + 2)
print(type(hours_text).__name__)
```

結果：32、5、`str`。文字列の `+` は文字をつなぎます。`int()` は整数への変換、`type()` は型の確認です。小数の文字列 "2.5" を数値にしたいときは `float()` を使います。

```
name = "学生 A"
hours = 2.5
print(f"{name}の学習時間は{hours}時間です")
```

結果：学生 A の学習時間は 2.5 時間です。先頭に `f` を付けた文字列では、`{}` に変数や計算式を入れられます。

`#` から行末はコメントです。処理の目的や単位をメモできます。

```
weekly_hours = 6 # 単位は時間／週
print(weekly_hours >= 5)
```

結果：`True`。`>=` は「以上」の比較です。

3.4.1 演習 2: 文字を数値にする

`minutes_text = "90"` を整数に変換し、60 で割って時間に直してください。`f` 文字列を使って 学習時間は 1.5 時間です と表示します。

3.5 リスト：複数の値をまとめる

5 人の 1 週間の学習時間を、リストにまとめます。角括弧 `[]` で囲み、値をコンマで区切ります。この章のデータはすべて架空です。

```
study_hours = [2, 5, 0, 3, 10]
print(study_hours[0])
print(study_hours[-1])
print(study_hours[1:3])
print(len(study_hours))
```

結果：2、10、[5, 0]、5。位置を表す番号（インデックス）は 0 から始まります。-1 は最後、[1:3] は 1 番目から 3 番目の直前までです。len() は要素数を返します。

```
total_hours = sum(study_hours)
mean_hours = total_hours / len(study_hours)
print(total_hours)
print(mean_hours)
print(min(study_hours), max(study_hours))
```

結果：20、4.0、0 10。sum() は合計、min() と max() は最小値・最大値です。平均は空のリストでは計算できません。

!0 と欠測を区別する

ここでの 0 は「学習時間が 0 時間という回答」です。「未回答」を 0 に置き換えると平均の意味が変わります。実際のデータ分析では、欠測をどう扱ったかを必ず記録します。

3.5.1 演習 3: 一つの値で平均はどう変わるか

study_hours を [2, 5, 0, 3, 20] に変えて平均を計算してください。最初の平均と比べ、平均だけで「典型的な学生」を説明する際の注意を 1 文で書きます。実験後は元の [2, 5, 0, 3, 10] に戻します。

3.6 条件分岐：条件で処理を変える

```
hours = 5
if hours >= 5:
    print("5 時間以上")
else:
    print("5 時間未満")
```

結果：5 時間以上。if の後ろに条件を書き、行末に : を付けます。条件が成り立つと、その下の字下げした処理が実行されます。else はそれ以外です。字下げは半角スペース 4 個にそろえます。

比較	意味
<code>a == b</code>	等しい
<code>a != b</code>	等しくない
<code>a > b</code> 、 <code>a >= b</code>	より大きい、以上
<code>a < b</code> 、 <code>a <= b</code>	より小さい、以下

代入の = と比較の == を区別してください。条件を組み合わせるときは and（両方）、or（少なくとも一方）を使います。

```
hours = 0
if hours == 0:
    print("0 時間")
elif hours < 5:
    print("0 時間より多く 5 時間未満")
else:
    print("5 時間以上")
```

結果：0 時間。elif は追加の条件です。上から調べ、最初に成り立った枝だけを実行します。この例は 0 以上の時間を前提にしています。

3.6.1 演習 4: 境界を確かめる

最初の if の例を、hours が 4、5、6 の場合で試してください。条件を hours > 5 に変えると、どの場合だけ結果が変わりますか。

3.7 繰り返し: 同じ処理を各データに行う

```
study_hours = [2, 5, 0, 3, 10]
for hours in study_hours:
    print(hours * 60)
```

結果（上から）: 120、300、0、180、600。for はリストから値を順に取り出して hours に代入し、字下げした処理を繰り返します。

3.7.1 条件に合う人数を数える

```
count = 0
for hours in study_hours:
    if hours >= 5:
        count = count + 1
print(count)
```

結果：2。count = count + 1 は、現在の値に 1 を足して代入し直します。count の初期化はループの前、最後の表示はループの外です。実行前に、2、5、0、3、10 を読んだ直後の count を紙に書いてみてください。

range(3) は 0、1、2 を順に生成します。決まった回数の繰り返しで使えます。

```
for i in range(3):
    print(i)
```

3.7.2 演習 5: 条件に合う値を集める

空のリスト selected = [] を用意し、5 時間以上の値を selected.append(hours) で追加してください。append() は末尾に要素を追加する操作です。元データを変更せず、結果が [5, 10] になることを確認します。

3.8 辞書: 項目名と値を対応させる

辞書は、項目名（キー）で値を取り出す仕組みです。リストの「何番目」と区別しましょう。

```
student = {"id": "A", "hours": 2}
print(student["id"])
print(student["hours"])
```

結果：A、2。辞書をリストに入れると、小さな表のように複数人の情報を扱えます。

```
students = [
    {"id": "A", "hours": 2},
    {"id": "B", "hours": 5},
    {"id": "C", "hours": 0},
]
for student in students:
    print(f"{student['id']}: {student['hours']}時間")
```

結果: A: 2時間、B: 5時間、C: 0時間。外側の二重引用符と内側の一重引用符を区別してください。大量の表データを扱う pandas は、この基礎の後で学びます。

3.9 関数: 処理に名前を付ける

同じ計算を別の値でも使えるよう、関数にまとめます。

```
def to_minutes(hours):
    return hours * 60

result = to_minutes(2.5)
print(result)
```

結果: 150.0。def は関数の定義、hours は受け取る値の名前 (引数)、return は返す値 (戻り値) の指定です。定義しただけでは計算は実行されません。to_minutes(2.5) と呼び出すと動きます。print() は表示、return は呼び出し元へ値を渡す操作です。返された値は次の計算に使えます。

```
print(to_minutes(1) + to_minutes(2))
```

結果: 180。関数内で return の代わりに print だけを使うと、呼び出しの戻り値は None (値がないことを表す値) になります。

3.9.1 演習 6: 割合を求める関数

percentage(part, whole) を定義し、part / whole * 100 を返してください。percentage(2, 5) が 40.0 になることを確認します。ここでは whole が正の数であることを前提にします。

3.10 エラーを読んで直す

エラーは、処理を進められなかった場所を知らせる情報です。まずメッセージの最後の行で種類を読み、示された行とその前後を確認します。

エラー	典型的な原因	最初に確かめること
NameError	変数が未定義	スペル、定義セルの実行
TypeError	型の組み合わせが不適切	文字列と数値の区別
SyntaxError	文法が不正	:, 括弧、引用符
IndentationError	字下げが不正	ブロックを半角 4 スペースにそろえる
IndexError	リストの範囲外	5 要素なら位置は 0~4
KeyError	辞書にキーがない	キーの名前
ZeroDivisionError	0 で割った	分母、空のリスト

修正問題: 以下は意図的に誤ったコードです。単独のセルで試し、理由を説明して直します。提出時には修正済みにしてください。

```
print("3" + 2)
```

もう一問: `study_hours = [2, 5, 0, 3, 10]` に対して `study_hours[5]` を使うとどうなるでしょう。最後の値を取り出す方法を二つ考えてください。

💡 AI に相談するとき

まず結果を予想し、自分で 1 回実行します。相談では「期待した結果」「実際の結果」「短いコード」を伝え、「答え全部ではなく、原因とヒントを説明して」と依頼してみましょう。修正後は自分で再実行し、変更理由を説明します。実在の学生情報や認証情報は入力しません。

3.11 総合課題：学習時間の集計メモ

次のデータを新しいセルに定義してください。これは小規模な架空データであり、実際の学生集団について推論するための標本ではありません。

```
survey_hours = [2, 5, 0, 3, 10]
```

1. 人数、合計、平均を計算し、単位とともに表示します。
2. 5 時間以上の人数と割合 (%) を求めます。人数は `for` と `if` で数え、割合は演習 6 の関数を使います。
3. Python 標準ライブラリの `statistics` から `median` を読み込み、中央値を求めます。標準ライブラリは Python に付属する道具で、追加インストールは不要です。
4. 平均と中央値の違い、0 時間の扱い、データの限界を Markdown セルに 3~5 文で説明します。
5. VS Code でカーネルを再起動し、上からすべて実行して同じ結果になることを確認します。

中央値の計算方法：

```
from statistics import median
print(median(survey_hours))
```

確認値： 5 人、20 時間、平均 4.0 時間、5 時間以上 2 人、40.0%、中央値 3 時間。

提出するものは、コード・結果・考察を含む `.ipynb` ファイルです。VS Code で保存し、作業フォルダ内のファイルを提出します。`.venv` フォルダは提出しません。環境確認セルの出力に PC のユーザー名などが含まれる場合は、その出力だけ消して保存してください。AI を利用した場合は、何を相談し、何を自分で確認したかを末尾に記録します。

確認観点 達成の目安

再現性	再起動後も上から実行できる
計算	全確認値が一致し、単位がある
理解	条件と繰り返しの動きを説明できる
解釈	0 と欠測を区別し、過度な一般化をしない

3.12 解答例と振り返り

先に自分のコードを保存してから比較してください。同じ結果になる別の書き方も正解です。

3.12.1 演習 1・2

```
book_price = 850
books = 3
shipping = 200
```

```
print(book_price * books + shipping)
books = 4
print(book_price * books + shipping)

minutes_text = "90"
hours = int(minutes_text) / 60
print(f"学習時間は{hours}時間です")
```

結果は 2750 円、3600 円、1.5 時間です。冊数を変えた後に合計を再計算しています。

3.12.2 演習 3・4

```
changed_hours = [2, 5, 0, 3, 20]
print(sum(changed_hours) / len(changed_hours))
```

平均は 6.0 時間です。一つの大きな値で平均が上がるため、分布や中央値も確認します。演習 4 で結果が変わるのは 5 時間のときです。>= は 5 を含み、> は含みません。

3.12.3 演習 5・6

```
study_hours = [2, 5, 0, 3, 10]
selected = []
for hours in study_hours:
    if hours >= 5:
        selected.append(hours)
print(selected)

def percentage(part, whole):
    return part / whole * 100

print(percentage(2, 5))
```

結果は [5, 10] と 40.0。修正問題は print(int("3") + 2) なら数値の 5 になります。文字の連結を意図するなら print("3" + "2") です。最後の要素は study_hours[4] または study_hours[-1] で取得できます。

3.12.4 総合課題の解答例

```
from statistics import median

def percentage(part, whole):
    return part / whole * 100

survey_hours = [2, 5, 0, 3, 10]
n = len(survey_hours)
total = sum(survey_hours)
count = 0
for hours in survey_hours:
    if hours >= 5:
        count = count + 1

print(f"人数: {n}人")
print(f"合計: {total}時間")
print(f"平均: {total / n:.1f}時間")
print(f"5 時間以上: {count}人")
print(f"割合: {percentage(count, n):.1f}%")
print(f"中央値: {median(survey_hours)}時間")
```

`:.1f` は小数点以下を 1 桁にそろえて表示する指定です。このコードは空でない数値リストを前提にしています。

考察例: 平均は 4.0 時間、中央値は 3 時間である。10 時間という値が平均を高くしている。0 時間は有効な回答として含めた。この架空の 5 人の結果を、大学生全体の傾向と考えることはできない。

振り返り: `=` と `==`、`print` と `return`、リストと辞書の違いを、コードを指しながら相手に説明してください。次は CSV の読み込みと pandas による表データの集計へ進みます。

3.13 公式資料

- [Python チュートリアル: 数値・文字列・リスト](#)
- [Python チュートリアル: 条件分岐・繰り返し・関数](#)
- [uv と Jupyter・VS Code の連携](#)
- [VS Code での Jupyter ノートブック](#)

教材確認日: 2026 年 10 月 6 日。画面の名称はソフトウェアの更新で変わることがあります。